Web Performance

A fast server is one step of many

LCP	INP	CLS
biggest element shown	click to next paint	layout jumps
2.5 s or less	200 ms or less	0.1 or less

Core Web Vitals: scored at the 75th percentile of real visits.

1 Hashed files

```
<script src="app.3f9a1c.js">
// response headers:
Cache-Control:
  max-age=31536000, immutable
// new build, new file name
// 2nd visit: no request sent
```

2 Below the fold

```
<img src="photo.jpg"
  loading="lazy"
  width="400" height="300">
// hero image: leave it eager
// width + height: no CLS jump
// lazy: loads when near view
```

measure DevTools Network + Performance, Lighthouse

TRAP What happens here?

```
// 1st visit to /app.js
HTTP/1.1 200 OK
Cache-Control: no-cache
ETag: "v1"
// 2nd visit, file unchanged
```

GET + If-None-Match

304, no body

stored copy reused

no-cache means store, but ask first.

KEY TAKEAWAY

- **measure** Before you fix
- **send less** Fewer bytes, less JS
- **cache** Reuse what you have

? Follow-up questions to expect

- **Which metric is about loading?** → LCP. INP is input latency, CLS is layout shift.
- **Does INP count scrolling?** → No. Only clicks, taps and key presses.
- **What is fetchpriority for?** → A hint to raise a resource's priority, e.g. the LCP image.
- **Lab or field data?** → Lab: Lighthouse runs. Field: real users, scored at p75.